



Executive Summary

Detailed Lecture Notes



Key Topics

This lecture focuses on **Graph Representation**, specifically how to store graph information within a computer system using appropriate data structures. It covers two primary methods: Adjacency Matrix and Adjacency List, exploring their application to different types of graphs (unweighted, weighted, undirected, and directed).



Important Definitions

- **Graph Representation:** The method used to store the vertices and edges of a graph in a computer's memory. This allows algorithms to efficiently process and analyze graph data.
- **Vertex (Node):** A fundamental unit of a graph, often represented by a number (e.g., 0, 1, 2, 3, 4).
- **Edge:** A connection between two vertices. Edges can be unweighted (simple connection) or weighted (have an associated cost or value).
- **Undirected Graph:** A graph where edges have no direction. If an edge exists between vertex A and B, it implies connectivity in both directions (A to B, and B to A).
- **Directed Graph:** A graph where edges have a specific direction. An edge from A to B does not necessarily imply an edge from B to A.
- **Unweighted Graph:** A graph where all edges have no associated cost or value; they simply represent a connection.
- **Weighted Graph:** A graph where each edge has an associated numerical value (weight), representing cost, distance, capacity, etc.



Adjacency Matrix

Concept

The Adjacency Matrix is a 2D array (matrix) where rows and columns represent vertices. For a graph with V vertices, the matrix will be $V \times V$ in size. Each cell `matrix[i][j]` stores information about the edge between vertex i and vertex j .

Representation Types

1. Unweighted, Undirected Graph:

- **Structure:** A $V \times V$ matrix, typically of boolean or integer type.
- **Logic:** If an edge exists between vertex i and j , `matrix[i][j]` is set to 1 (or true). Otherwise, it's 0 (or false).
- **Symmetry:** Since it's undirected, if `matrix[i][j]` is 1, then `matrix[j][i]` must also be 1.
- **Self-loops:** `matrix[i][i]` is typically 0 unless self-loops are allowed.

2. Weighted, Undirected Graph:

- **Structure:** A $V \times V$ matrix of integer type (or float/double for non-integer weights).
- **Logic:** If an edge exists between vertex i and j with weight W , `matrix[i][j]` is set to W . If no edge exists, it's 0 (or infinity in some implementations to distinguish from zero-weight edges).
- **Symmetry:** If `matrix[i][j]` is W , then `matrix[j][i]` must also be W .

3. Unweighted, Directed Graph:

- **Structure:** A $V \times V$ matrix, typically of boolean or integer type.
- **Logic:** If a directed edge exists from vertex i to j , `matrix[i][j]` is set to 1. Otherwise, it's 0.
- **Asymmetry:** `matrix[i][j]` being 1 does not imply `matrix[j][i]` is 1. They are independent.

4. Weighted, Directed Graph:

- **Structure:** A $V \times V$ matrix of integer type.
- **Logic:** If a directed edge exists from i to j with weight W , `matrix[i][j]` is set to W . If no edge, it's 0.
- **Asymmetry:** Similar to unweighted directed graphs, `matrix[i][j]` and `matrix[j][i]` are independent.

Implementation (C++ using `std::vector`)

- A `std::vector<std::vector<bool>>` or `std::vector<std::vector<int>>` can be used to represent the 2D matrix.
- Initialization: All cells are typically initialized to 0 (or false).
- Adding an edge (u, v):
 - For unweighted undirected: `adjMatrix[u][v] = 1; adjMatrix[v][u] = 1;`
 - For weighted undirected: `adjMatrix[u][v] = weight; adjMatrix[v][u] = weight;`
 - For unweighted directed: `adjMatrix[u][v] = 1;`

For unweighted undirected graphs: $\text{adjMatrix}[u][v] = 1$;

- For weighted directed: $\text{adjMatrix}[u][v] = \text{weight}$;

Time and Space Complexity

- **Space Complexity:** $O(V^2)$ because it requires a $V \times V$ matrix, regardless of the number of edges. This can be inefficient for sparse graphs (graphs with few edges).
- **Time Complexity (for building):** $O(V^2)$ to initialize the matrix, plus $O(E)$ for adding E edges. In practice, often considered $O(V^2)$ if initialization dominates or if iterating through all possible edges is required.



Adjacency List

Concept

The Adjacency List represents a graph as an array of lists (or vectors). Each index in the array corresponds to a vertex, and the list at that index contains all the vertices adjacent to it (i.e., its neighbors).

Representation Types

1. Unweighted, Undirected Graph:

- **Structure:** An array where each element is a dynamic list (e.g., `std::vector<int>` in C++ or `LinkedList` in Java).
- **Logic:** For each vertex i , $\text{adjList}[i]$ stores a list of all vertices j such that there is an edge between i and j .
- **Adding an edge (u, v):** $\text{adjList}[u].\text{push_back}(v)$; $\text{adjList}[v].\text{push_back}(u)$;

2. Weighted, Undirected Graph:

- **Structure:** An array where each element is a list of pairs (neighbor, weight).
- **Logic:** For each vertex i , $\text{adjList}[i]$ stores a list of pairs (j, W) where j is a neighbor and W is the weight of the edge (i, j) .
- **Adding an edge (u, v, W):** $\text{adjList}[u].\text{push_back}(\{v, W\})$;
 $\text{adjList}[v].\text{push_back}(\{u, W\})$;

3. Unweighted, Directed Graph:

- **Structure:** Same as unweighted undirected, but only add for the direction of the edge.
- **Adding an edge (u, v):** $\text{adjList}[u].\text{push_back}(v)$; (no entry for v to u unless a separate edge exists).

4. Weighted, Directed Graph:

- **Structure:** Same as weighted undirected, but only add for the direction of the edge.
- **Adding an edge (u, v, W):** $\text{adjList}[u].\text{push_back}(\{v, W\})$;

Implementation (C++ using `std::vector`)

- A `std::vector<std::vector<int>>` for unweighted graphs (where the inner vector stores neighbors).

- A `std::vector<std::vector<std::pair<int, int>>>` for weighted graphs (where the inner vector stores pairs of {neighbor, weight}).
- Initialization: The outer vector is sized to V , and inner vectors are initially empty.
- Adding an edge (u, v) for unweighted undirected: `adjList[u].push_back(v);`
`adjList[v].push_back(u);`

Time and Space Complexity

- **Space Complexity:** $O(V + E)$ where V is the number of vertices and E is the number of edges. This is generally more efficient for sparse graphs than Adjacency Matrix because it only stores existing edges.
- **Time Complexity (for building):** $O(V)$ to initialize the array of lists, plus $O(E)$ for adding E edges. Total $O(V + E)$.



Takeaways

- The choice between Adjacency Matrix and Adjacency List depends on the specific graph properties (dense vs. sparse) and the operations to be performed.
- Adjacency Matrix is better for dense graphs (many edges) and when checking for the existence of an edge between two specific vertices ($O(1)$ lookup) is frequent.
- Adjacency List is better for sparse graphs (few edges) and when iterating over all neighbors of a vertex ($O(\text{degree}(V))$) is frequent. It also uses less memory for sparse graphs.

